Applied Programming

Week 7: Review

Connecting concepts and coding skills from Weeks 1–6

School of Mechanical Engineering, Soongsil University
2026-2 Applied Programming

A useful program does more than produce a number. It gives inputs a clear meaning, performs a calculation, checks its result and keeps related data consistent. This week revisits the tools used to do those jobs. The examples are for review, not a preview of the examination questions.

1 Run, inspect and save a program

Review connection: Week 1, Sections 1 and 4–10.

A **program** is a set of instructions. The Python **interpreter** executes those instructions. A **virtual environment** keeps a project's interpreter and installed packages separate from other projects. Activating an environment selects which Python and packages a command uses. It does not change the folder or correct errors in a calculation.

Prepare the supplied review files

Extract `week07_review_examples.zip`. Work in its `week07_review` folder, outside your earlier assignment repositories. All five example files and their tests are complete. They do not call unfinished functions from your previous assignments. Use the existing Week 6 environment. A GPU and a new package installation are not needed.

Run – Anaconda Prompt

Replace the example folder below with the folder you extracted.

```
conda activate applied-programming-w06
cd /d "C:\your-course-folder\week07_review"
dir
python -c "import sys; print(sys.executable)"
python functions_review.py
```

The directory listing must include `functions_review.py`. The interpreter path must point to the activated environment. On macOS or Linux, use `cd "/your/folder/week07_review"` and `ls` instead of `cd /d` and `dir`. The Python commands are the same. Enter commands in a terminal, not after Python's `>>>` prompt. Each complete demonstration prints several sections in order. Each Output box below shows the part produced by its adjacent code.

An environment check verifies the tools available to run a program. A calculation test checks a result against a stated expectation. An environment can therefore work correctly while a calculation test fails. In the following sections, read the input and expected meaning before interpreting a test result.

Files, imports and saved revisions

A **module** is a Python file that can be imported. A **package** groups related modules. The review files use `if __name__ == "__main__":` to run their demonstration when executed directly. Importing their functions or classes does not run that guarded demonstration.

Git records versions of files. `git diff` shows unstaged edits, `git add` stages the current content, and `git commit` records the staged content locally. `git push` transfers commits to a remote repository. Saving in an editor, committing and pushing are different actions. If a file changes after staging, stage the new change again to include it in the next commit. This review ZIP does not need to become a new repository.

2 Write functions with checkable results

Review connection: Week 1, Sections 8–9. Week 2, Sections 2.1–2.3.

2.1 Follow the value from input to return

A **function** names a reusable calculation or operation. Its parameters name the inputs inside the function. Arguments are the values supplied by a call. The **caller** is the code making that call, not a different kind of function.

The input below is a length in millimetres. The function accepts non-negative Python `int` or `float` values that convert to a finite `float`, excluding Boolean values. It returns metres as a `float` without modifying the input. The conversion rule is metres = millimetres/1000. The checks make invalid input explicit instead of silently replacing it with zero. The file already imports `math` for `math.isfinite`.

Code – functions_review.py, function definition

```
def mm_to_m(value_mm):
    """Convert a finite, nonnegative Python int or float from mm to m."""
    if isinstance(value_mm, bool) or not isinstance(value_mm, (int, float)):
        raise ValueError("length must be a real number, not Boolean")
    try:
        value = float(value_mm)
    except OverflowError as error:
        raise ValueError("length must be finite") from error
    if not math.isfinite(value) or value < 0.0:
        raise ValueError("length must be finite and nonnegative")
    return value / 1000.0
```

Code – the same file, demonstration calls

```
length_m = mm_to_m(12.5)
print("returned length:", length_m)
print("zero length:", mm_to_m(0.0))
try:
    mm_to_m(-1.0)
except ValueError as error:
    print("rejected:", error)
```

Output

```
returned length: 0.0125
zero length: 0.0
rejected: length must be finite and nonnegative
```

`return` sends a value back to the caller. The assignment stores that value in a name. `print` displays a value on the screen. A function that only prints and reaches its end without `return` returns `None`, not the text that appeared on the screen. The two calls above use the same rule with an ordinary input and the boundary value zero.

2.2 Use containers and loops without losing the meaning

A **tuple** is an ordered sequence whose elements cannot be reassigned. A **list** is an ordered sequence that can be changed. Both use zero-based indexing. A slice `values[1:3]` selects indices 1 and 2, excluding the stop index 3. Unpacking assigns the elements of a sequence to separate names.

Code – functions_review.py, several measurements

```
start = (2.0, 4.0)
x, y = start
readings_mm = [0.0, 12.5]
readings_mm.append(25.0)
converted = []
for value in readings_mm:
    converted.append(mm_to_m(value))
compact = [mm_to_m(value) for value in readings_mm]
print("unpacked:", x, y)
print("first two:", readings_mm[:2])
print("converted:", converted)
print("same values:", converted == compact)
```

Output

```
unpacked: 2.0 4.0
first two: [0.0, 12.5]
converted: [0.0, 0.0125, 0.025]
same values: True
```

The loop and the list comprehension apply the same function to each input. A comprehension is a compact expression for constructing a collection. It is not a different numerical method. The input units remain millimetres and the new list contains metres.

2.3 Test the contract, not just one convenient value

A **contract** states accepted inputs, the meaning of the result and how invalid input is handled. A **test** calls the program and checks a specific expectation. An **assertion** fails when the condition it checks is false. Type annotations such as `-> float` describe an intended return type. They do not perform input validation at runtime.

Code – test_review.py, supplied checks

The file already imports `pytest` and `mm_to_m` from `functions_review`.

```
def test_mm_to_m_conversion():
    assert mm_to_m(12.5) == pytest.approx(0.0125, rel=1e-12, abs=1e-12)
    assert mm_to_m(0.0) == 0.0
```

```
def test_mm_to_m_negative_length():
    with pytest.raises(ValueError):
        mm_to_m(-1.0)
```

`pytest.raises` checks that a call rejects invalid input with the expected exception. It does not turn the invalid input into an accepted result. These tests are supplied so that review time can focus on explaining what they check.

Run – Anaconda Prompt, week07_review

```
python -m pytest -q test_review.py -k mm_to_m
```

Output

```
...........                                                              [100%]
11 passed, 16 deselected in 0.04s
```

`-k` selects test names matching the given expression. `deselected` means not selected, not failed. The full suite is run without `-k`. Use `-x` when you want execution to stop at the first failure. In a failure report, `E` introduces error detail. Read the test name, call and actual value before changing code. `NotImplementedError` indicates unfinished code. An assertion failure means an expectation did not hold after the code ran.

Floating-point results and tolerance

The file also imports `numpy as np`. For finite values, `np.isclose(actual, expected, rtol=..., atol=...)` uses

$$|\text{actual} - \text{expected}| \leq \text{atol} + \text{rtol}\,|\text{expected}|.$$

The **absolute tolerance** `atol` sets an allowance in the result's units. The **relative tolerance** `rtol` scales an allowance with the magnitude of the reference value. Near zero, the relative term contributes little or nothing, so `atol` matters. Tolerance allows a stated numerical difference, not an incorrect formula. As in Week 2, `pytest.approx(expected, rel=..., abs=...)` accepts a difference within either the absolute or relative allowance. Unlike `np.isclose`, it does not add the two allowances.

Code – functions_review.py, tolerance comparison

```
print(np.isclose(100.05, 100.0, rtol=0.001, atol=1e-6))
print(np.isclose(0.0001, 0.0, rtol=0.001, atol=1e-6))
```

Output

```
True
False
```

For the first comparison, the difference is approximately 0.05 and the allowed difference is 0.100001. Near the zero reference, the same relative tolerance does not admit an arbitrary nonzero value. Always choose tolerances deliberately rather than changing them merely to make a test pass.

3 Read NumPy arrays by shape and axis

Review connection: Week 2, Sections 2.4–2.6.

3.1 Shape tells us what each position represents

NumPy provides arrays and operations over many numerical values. An array's **shape** gives the size of each dimension. Its **dtype** describes the element representation. Here, three rows contain three measured two-dimensional positions in millimetres. This is a review data array, not a call to a previous assignment's path API.

Run – Anaconda Prompt, week07_review

```
python arrays_review.py
```

Code – arrays_review.py, indexing

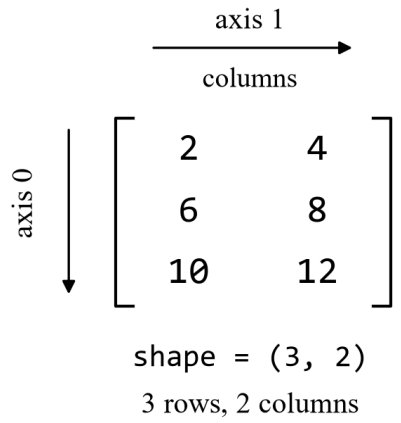
```
points = np.array([[2., 4.], [6., 8.], [10., 12.]])
print("shape:", points.shape)
print("second x:", points[1, 0])
print("row / one-row slice:", points[1].shape, points[1:2].shape)
```

Output

```
shape: (3, 2)
second x: 6.0
row / one-row slice: (2,) (1, 2)
```

`points[1]` selects one row and has shape `(2,)`. `points[1:2]` selects a range containing one row and keeps shape `(1, 2)`. These contain the same two numbers but have different dimensions. The supplied file imports `numpy as np`.

One array, two reduction directions



`points.sum(axis=0)`

Combine the rows, keep one total per column.

`[18, 24]`

`points.sum(axis=1)`

Combine the columns, keep one total per row.

`[6, 14, 22]`

Figure 7.1: Each reduction removes the selected axis. The surviving axis identifies the meaning of each output value.

Code – arrays_review.py, reductions

```
print("sum each row:", np.sum(points, axis=1))
print("sum each coordinate:", np.sum(points, axis=0))
```

Output

```
sum each row: [ 6. 14. 22.]
sum each coordinate: [18. 24.]
```

An **axis** is a dimension of the array. `axis=0` combines the rows and leaves one total per coordinate column. `axis=1` combines the columns and leaves one total per row. A sum without an axis combines every element into one scalar. The appropriate axis follows the question being answered, not the desired number of output values.

3.2 Broadcasting and ownership are different questions

Broadcasting applies compatible array operations without manually repeating smaller arrays. Adding a shape `(2,)` displacement to shape `(3, 2)` positions adds the same two coordinate offsets to every row. Matching proceeds from the trailing dimensions: sizes must match or one of them must be 1.

Code – arrays_review.py, translation

```
shift = np.array([1., -2.])
translated = points + shift
print("translated:")
print(translated)
```

Output

```
translated:
[[ 3.  2.]
 [ 7.  6.]
 [11. 10.]]
```

The expression creates new results. It does not mean that every way of obtaining another array creates independent storage. Basic slicing commonly returns a **view**, which shares data with its source. An explicit **copy** has independent storage.

Code – arrays_review.py, view and copy

```
shared_source = points.copy()
view = shared_source[1:2]
view[0, 0] = 99.0
independent_source = points.copy()
copied = independent_source[1:2].copy()
copied[0, 0] = 99.0
print("after view edit:", shared_source[1])
print("after copy edit:", independent_source[1])
```

Output

```
after view edit: [99.  8.]
after copy edit: [6. 8.]
```

The first edit reaches the original through a view. The later edit changes only the copy. Assignment such as `other = points` creates another reference, not a copy of the array. Before modifying a value, identify which object owns the data and whether another name can reach it.

From one array to a batch

A **batch** adds a dimension for multiple independent examples. Shape `(2, 3, 2)` means two groups, three positions per group and two coordinates per position. `np.sum(batch, axis=1)` sums the positions within each group. Shape `(2, 2)` retains two coordinate totals per group.

Code – arrays_review.py, batch dimensions

```
batch = np.array([points, translated])
print("batch shape:", batch.shape)
print("first point from each set:")
print(batch[:, 0, :])
totals = np.sum(batch, axis=1)
print("total shape:", totals.shape)
print(totals)
```

Output

```
batch shape: (2, 3, 2)
first point from each set:
[[2. 4.]
 [3. 2.]]
total shape: (2, 2)
[[18. 24.]
 [21. 18.]]
```

Check both the numerical values and the shape. An accidentally removed batch dimension can mix independent examples even when the output looks plausible.

4 Use objects to keep data and behaviour together

Review connection: Week 3, Sections 2–6.

4.1 A class defines the behaviour of its instances

A **class** defines a type of object. An **instance** is a particular object created from that class. An **attribute** is data accessed through the object. A **method** is a function defined on the class and used with an instance. In an instance method, `self` refers to that instance.

The complete `ReviewObstacle` class below stores one circular obstacle's name, centre and positive radius. Unlike Week 4's frozen `Obstacle`, this separate teaching type permits a controlled radius change. It does not replace that API. Centres are valid two-coordinate tuples in metres. Follow three operations: `__init__` stores the inputs, `radius` reads the stored value, and `resize` checks a new value before storing it. The file imports `math` and defines `RadiusError` as a specific kind of `ValueError`, discussed further in Section 4.3.

```
class RadiusError(ValueError):
    """An invalid radius is also an invalid value."""
```

Code – objects_review.py, supplied class

```
class ReviewObstacle:
    def __init__(self, name, center, radius):
        self.name = name
        self.center = tuple(center)
        self.resize(radius)

    @property
    def radius(self):
        return self._radius

    def resize(self, new_radius):
        if (isinstance(new_radius, bool)
                or not isinstance(new_radius, (int, float))):
            raise RadiusError("radius must be a real number, not Boolean")
        try:
            value = float(new_radius)
        except OverflowError as error:
            raise RadiusError("radius must be finite and positive") from error
        if not math.isfinite(value) or value <= 0.0:
            raise RadiusError("radius must be finite and positive")
        self._radius = value
```

`__init__` initializes an instance after it is created. A call such as `ReviewObstacle("post", (1.0, 2.0), 0.5)` supplies the values for that initialization. `__init__` does not return the new object itself.

The `radius` **property** exposes a computed or controlled read through ordinary attribute syntax. Read it as `first.radius`, without parentheses. Call the method as `first.resize(0.8)`. Writing `first.resize` without parentheses retrieves the callable method but does not perform the resize.

Run – Anaconda Prompt, week07_review

```
python objects_review.py
```

Code – objects_review.py, calls and state changes

```
first = ReviewObstacle("post", (0.0, 0.0), 0.5)
second = ReviewObstacle("post", (0.0, 0.0), 0.5)
alias = first
alias.resize(0.75)
print("same object:", alias is first)
print("separate objects:", first is second)
print("radii:", first.radius, second.radius)
try:
    first.resize(0.0)
except ValueError as error:
    print("rejected:", error)
print("radius after rejection:", first.radius)
```

Output

```
same object: True
separate objects: False
radii: 0.75 0.5
rejected: radius must be finite and positive
radius after rejection: 0.75
```

A valid resize changes one object's stored radius. A rejected resize leaves its previous radius intact because validation occurs before assignment. The class keeps related data and the operations that maintain its rules together. The leading underscore in `_radius` marks internal use by convention, not a security barrier.

Class attributes belong to the class. Instance attributes such as `first.name` and `first._radius` belong to a particular instance. A mutable list intended for one instance must be created for that instance, not shared as one class-level list. This is the distinction shown by Week 3's `BadPlanningRun` example.

4.2 Dataclasses reduce repeated initialization code

`dataclasses` is a Python standard-library module. The supplied file uses `from dataclasses import dataclass`. `@dataclass` generates routine methods, including an initializer for the declared fields. It does not automatically decide which coordinates are valid.

Code – objects_review.py, a small coordinate value

```
@dataclass(frozen=True)
class Point:
    values: tuple

    def __post_init__(self):
        owned = tuple(self.values)
        if not owned:
            raise ValueError("values must not be empty")
        object.__setattr__(self, "values", owned)

    @property
    def count(self):
        return len(self.values)
```

The generated initializer assigns `values`, then calls `__post_init__`. The latter converts the supplied sequence to a tuple and rejects an empty sequence. The exact double-underscore name is important: a method named `postinit` would not be called automatically. `object.__setattr__` is used here during initialization to store the normalized value in the frozen instance.

Code – the same file, ownership check

```
source = [2.0, 4.0]
point = Point(source)
source[0] = 99.0
print("source:", source)
print("stored:", point.values)
print("count:", point.count)
print("equal fields:", point == Point((2.0, 4.0)))
```

Output

```
source: [99.0, 4.0]
stored: (2.0, 4.0)
count: 2
equal fields: True
```

`frozen=True` prevents ordinary field reassignment. It is not **deep immutability**: a mutable list or array held inside a frozen instance can still be changed. This example stores a tuple of scalar numbers, so changing the original list does not change the stored coordinates. A tuple containing mutable objects would still refer to those objects.

An **invariant** is a condition that must remain true for a valid object. Here, the point has at least one coordinate and the obstacle radius is positive. In Week 3, related objects also had to agree, for example on their number of coordinates. A type annotation is not a substitute for those checks.

4.3 Composition and inheritance express different relationships

Composition builds an object using other objects. A scene holds obstacles rather than pretending to be an obstacle. The supplied example stores a new list containing the given obstacle references.

Code – objects_review.py, composition

```
class Scene:
    def __init__(self, obstacles):
        self.obstacles = list(obstacles)
```

```
scene = Scene([first, second])
print("scene count:", len(scene.obstacles))
print("first part is first:", scene.obstacles[0] is first)
```

Output

```
scene count: 2
first part is first: True
```

The list and the objects in it have different roles. Copying the list protects its membership from changes to the caller's list. It does not duplicate the obstacles. The next section follows those references explicitly.

Inheritance defines a more specific type from an existing type. As with Week 3's error class, `class RadiusError(ValueError)` makes the new error a subclass of `ValueError`. Catching `ValueError` therefore catches `RadiusError`. This is a type relationship, not the containment used in composition.

5 Choose collections and understand their costs

Review connection: Week 4, Sections 2–3.

5.1 Choose the operation before the container

Objects describe individual items. **Collections** organize several items so that a program can visit them in order, look them up by identifier, detect duplicates or process requests. The following file imports the complete `ReviewObstacle` class from `objects_review.py`. It does not depend on your Week 3 assignment.

Run – Anaconda Prompt, week07_review

```
python collections_review.py
```

Code – collections_review.py, list ownership

```
post = ReviewObstacle("post", (0.0, 0.0), 0.5)
second = ReviewObstacle("second", (1.0, 0.0), 0.4)
marker = ReviewObstacle("marker", (2.0, 0.0), 0.25)
obstacles = [post, second]
copied = list(obstacles)
print("new list:", copied is not obstacles)
print("same first object:", copied[0] is obstacles[0])
copied[0].resize(0.75)
copied.append(marker)
print("list lengths:", len(obstacles), len(copied))
print("original first radius:", obstacles[0].radius)
```

Output

```
new list: True
same first object: True
list lengths: 2 3
original first radius: 0.75
```

Different list containers, shared original objects

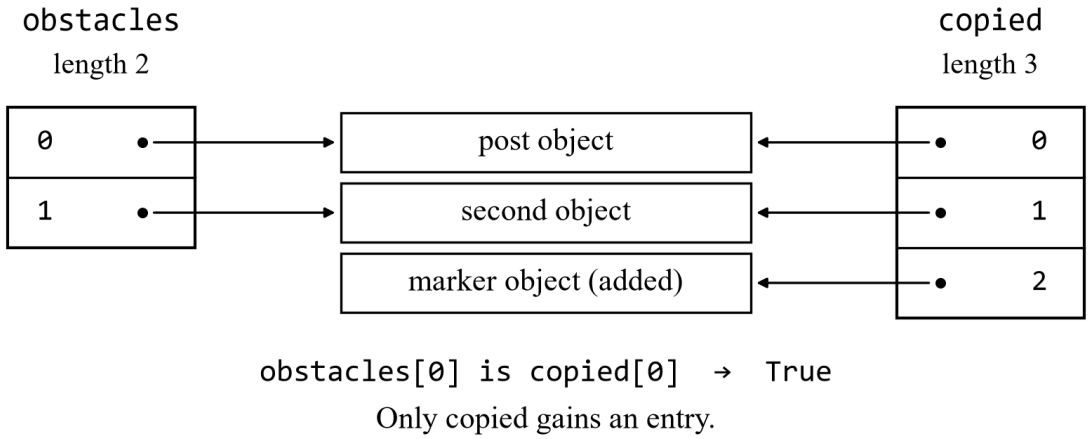


Figure 7.2: A shallow list copy creates a new container but preserves references to its existing elements. Adding an element to one list does not add it to the other.

`is` checks whether two references reach the same object. It does not compare numerical content. If the shared obstacle is resized, both lists still refer to that changed object.

Code – collections_review.py, lookup and duplicates

```
by_name = {item.name: item for item in obstacles}
print("lookup second:", by_name["second"].center)
names = ["post", "second", "post"]
seen = set()
duplicates = []
for name in names:
    if name in seen:
        duplicates.append(name)
    seen.add(name)
print("duplicates:", duplicates)
```

Output

```
lookup second: (1.0, 0.0)
duplicates: ['post']
```

A **dictionary** maps keys to values. Here, an identifier is a key and the obstacle is its value. A **set** stores distinct values and supports membership tests. Here it tracks string IDs already seen. It does not replace an ordered request sequence. Dictionary construction alone does not reject duplicate IDs: a later value for the same key replaces the earlier value. This example's `duplicates` list records later occurrences. Week 4 used a set to keep each repeated ID only once.

5.2 FIFO and LIFO are processing policies

A **deque** allows addition and removal at either end. The file imports it with `from collections import deque`. Appending on the right and removing on the left with `popleft()` implements **FIFO**, first in, first out. Appending and removing on the right with `pop()` implements **LIFO**, last in, first out. The choice of operations determines the policy.

Code – collections_review.py, same inputs and different removals

```
fifo = deque(["inspect", "place", "reset"])
lifo = deque(["inspect", "place", "reset"])
print("FIFO removed:", fifo.popleft())
print("FIFO remaining:", list(fifo))
print("LIFO removed:", lifo.pop())
print("LIFO remaining:", list(lifo))
```

Output

```
FIFO removed: inspect
FIFO remaining: ['place', 'reset']
LIFO removed: reset
LIFO remaining: ['inspect', 'place']
```

FIFO serves the oldest stored request first in this use. LIFO serves the newest first. Neither behaviour is forced by the name `deque` alone.

5.3 Big-O describes growth, not a stopwatch reading

Computational complexity relates work to input size. **Big-O notation** describes an upper growth class as input size becomes large. Here, n is the number of stored elements. Constant-time $O(1)$ means the bound does not grow with n, not that an operation takes one machine instruction or no time.

The table compares operations you have just used. Read both the operation and the condition before comparing costs.

Operation	Cost	Condition
Find a value in a list	$O(n)$	Worst case: scan the list
Set membership / dictionary lookup	$O(1)$	Average case, worst case $O(n)$
Append to a list	$O(1)$	Amortized over many appends
Remove first list item	$O(n)$	Shift remaining references
Remove first deque item	$O(1)$	`popleft()`
Drain a list from its front	$O(n^2)$	Repeat front removal
Drain a deque from its front	$O(n)$	Repeat `popleft()`

For a four-element list, repeated front removal shifts $3 + 2 + 1 + 0 = 6$ references. A deque needs four constant-time removals instead. One removal and draining the entire container are different tasks.

Average cost describes the usual case under assumptions about inputs or hashing. **Amortized** cost spreads occasional expensive operations over a sequence. For example, a list may occasionally resize, but many appends together have constant cost per append on an amortized basis.

Other growth patterns also recur. Halving a remaining search range gives logarithmic growth, $O(\log n)$. Visiting each item gives linear growth, $O(n)$. Efficient comparison sorting commonly has $O(n \log n)$ cost. Comparing every pair can require quadratic work, $O(n^2)$. These growth classes do not determine which operation is fastest for a small input. A benchmark measures one implementation and machine under stated conditions, not a universal speed ratio.

6 Save values without confusing text and objects

Review connection: Week 4, Section 4.

JSON is a text format for exchanging structured values. **Serialization** converts values into a representation that can be stored or transmitted. Parsing reads that representation back. A JSON string, a Python dictionary and an obstacle instance are three different things.

The final part of `collections_review.py` uses one obstacle record. It converts the dictionary into JSON text, writes the text to a file, reads it back and explicitly constructs an object from the recovered values. The file imports `json` and `Path` from `pathlib`. This `Path` is a filesystem path, not the geometric path class from Week 3.

Code – collections_review.py, JSON round trip

```
record = {"name": marker.name,
          "center": list(marker.center), "radius": marker.radius}
text = json.dumps(record, allow_nan=False)
print("JSON text:", text)
output = Path("outputs/obstacle.json")
output.parent.mkdir(exist_ok=True)
output.write_text(text, encoding="utf-8")
loaded = json.loads(output.read_text(encoding="utf-8"))
restored = ReviewObstacle(
    loaded["name"], loaded["center"], loaded["radius"]
)
print("loaded type:", type(loaded).__name__)
print("restored:", restored.name, restored.center, restored.radius)
```

Output

```
JSON text: {"name": "marker", "center": [2.0, 0.0], "radius": 0.25}
loaded type: dict
restored: marker (2.0, 0.0) 0.25
```

Generated file

`outputs/obstacle.json` contains the JSON text below the current review folder. Open it in your editor to inspect the same fields. The path is a filename, not a terminal command.

`json.dumps` returns a string. `json.loads` returns Python values. A JSON array becomes a Python list, not an instance of a course class. The explicit `ReviewObstacle(...)` call restores the object and applies its radius check. `encoding="utf-8"` uses the same text encoding when writing and reading. No encoding algorithm needs to be implemented.

Week 4's scene loader distinguishes three validation stages:

- **Parse error**: invalid JSON text, such as a missing closing brace.
- **Schema error**: invalid fields, types or permitted numerical ranges. The Week 4 schema rejects a missing radius or `radius_m: 0`.
- **Domain error**: a relationship violates the scene rules, such as a repeated obstacle ID or a start outside the bounds.

Our short example assumes valid fields and types. Its direct `ReviewObstacle(...)` call runs the constructor's own radius check, not Week 4's loader or schema checks. In that loader, schema validation precedes object construction. JSON text should contain finite numbers. `allow_nan=False` rejects non-finite values when writing. Successful parsing alone does not establish a usable scene.

7 Separate parameters, measurements and decisions

Review connection: Week 5, Sections 1–4. Week 6, Section 2.

A **parametric model** uses a fixed calculation rule with named inputs that can be varied. A parameter changes a model setting. A measured or computed response describes the result. An **objective** says what to improve, such as reducing length. A **constraint** states what is allowed, such as maintaining sufficient clearance.

7.1 More samples do not create a different route

Interpolation places points between given values. For two endpoint vectors, $p(t) = (1 - t)p_0 + tp_1$, with $0 \leq t \leq 1$, stays on the straight segment. It does not find a detour or specify motion speed without a time interpretation.

Here the route goes from $(-2, 0)$ to $(2, 0)$ metres. A circular obstacle is centred at the origin with radius 0.5 m. For a sampled point p, the signed surface clearance is $\|p\| - 0.5$. The supplied model checks sampled points, not the entire continuous segment. In the code, `[:, None]` changes the interpolation weights from shape `(count,)` to `(count, 1)`. Broadcasting then applies each weight to both coordinates, producing `(count, 2)` points.

Run – Anaconda Prompt, week07_review

```
python models_review.py
```

Code – models_review.py, same endpoints and two sample counts

```
start = np.array([-2.0, 0.0])
goal = np.array([2.0, 0.0])
radius = 0.5
for count in (2, 9):
    alpha = np.linspace(0.0, 1.0, count)[:, None]
    points = (1.0 - alpha) * start + alpha * goal
    distances = np.linalg.norm(points, axis=1)
    clearance = float(np.min(distances - radius))
    print(f"samples={count}: sampled clearance={clearance:.1f} m")
```

Output

```
samples=2: sampled clearance=1.5 m
samples=9: sampled clearance=-0.5 m
```

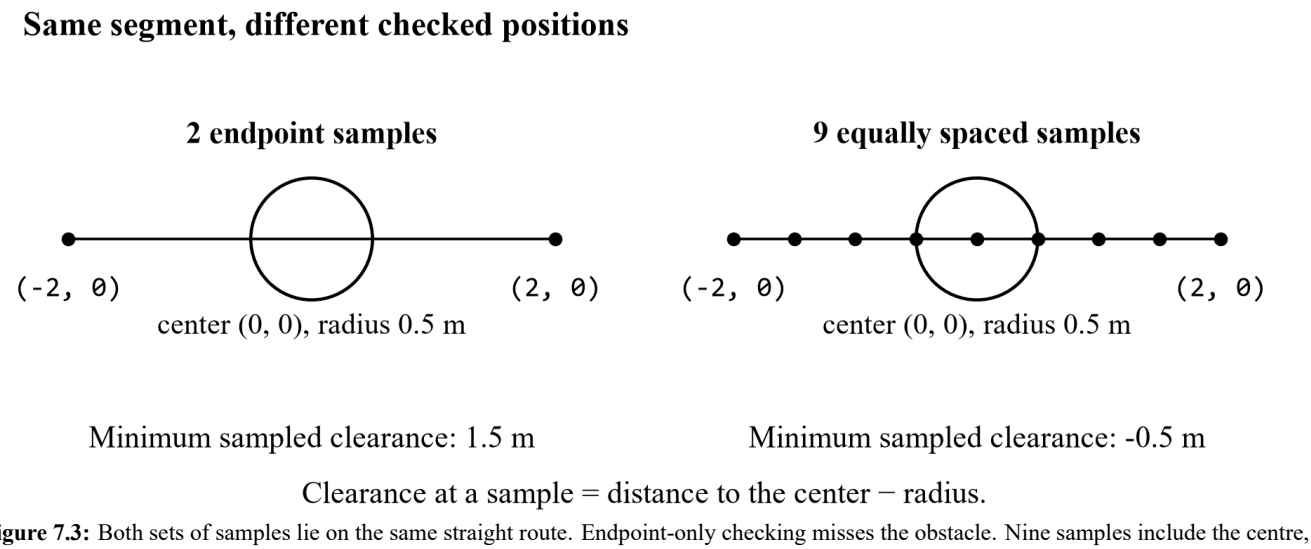


Figure 7.3: Both sets of samples lie on the same straight route. Endpoint-only checking misses the obstacle. Nine samples include the centre, where signed clearance is negative.

The geometry did not improve when the number of samples increased. The check became better able to detect this particular violation. Even a finite set of non-colliding samples does not generally prove that every point between them is collision-free. Week 6 distinguished sampled checks from whole-segment checks.

7.2 Changing a requirement does not change a measurement

For this separate threshold example, suppose another checked route has clearance 1.5 m. This is not the preceding nine-sample result of −0.5 m. Define margin as measured clearance minus required clearance. The requirement is satisfied when the margin is non-negative, including equality.

Code – models_review.py, threshold comparison

```
measured_clearance = 1.5
for required_clearance in (0.5, 1.5, 1.6):
    margin = measured_clearance - required_clearance
    print(f"required={required_clearance:.1f}: "
          f"margin={margin:.1f}, accepted={margin >= 0.0}")
```

Output

```
required=0.5: margin=1.0, accepted=True
required=1.5: margin=0.0, accepted=True
required=1.6: margin=-0.1, accepted=False
```

All three rows use the same measurement. The final row is rejected because the required clearance is larger. For an upper length limit, the corresponding margin is *limit minus measured length*, so non-negative still means allowed. Compare the unrounded values. Formatting an output to one decimal place must not change the value used in the decision. Rejection is a normal result for a valid input that violates a constraint. It is different from raising an exception because the input itself is invalid.

8 Review grid search and greedy improvement

Review connection: Week 6, Sections 2–4.

8.1 A grid enumerates combinations of parameter values

Grid search evaluates combinations from specified parameter values. Three x values and three y values for an intermediate waypoint generate nine coordinate pairs, not three.

Code – models_review.py, Cartesian combinations

```
xs = np.linspace(-1.0, 1.0, 3)
ys = np.linspace(-1.0, 1.0, 3)
pairs = [(float(x), float(y)) for x in xs for y in ys]
print("x values:", xs)
print("y values:", ys)
print("grid count:", len(pairs))
print("first / last:", pairs[0], pairs[-1])
```

Output

```
x values: [-1.  0.  1.]
y values: [-1.  0.  1.]
grid count: 9
first / last: (-1.0, -1.0) (1.0, 1.0)
```

The inner loop runs for every outer-loop value. Pairing corresponding indices would miss six combinations. With k values per dimension, two dimensions give k^2 pairs and d dimensions give k^d combinations.

A finer grid adds evaluations and may find a better feasible result, but does not guarantee the continuous optimum. Each candidate must still pass the constraints. Generating coordinates alone does not complete the search or make a rejected result acceptable.

8.2 Greedy improvement makes one local decision, then reevaluates

A **greedy algorithm** makes a locally preferred choice at the current step. Week 6's route-shortening rule keeps the endpoints and the original input path. It selects the allowed interior-point deletion with the largest saving strictly greater than 10^{-12} m. On an exact tie, it keeps the first qualifying point in the current path order.

For three consecutive points A, B, C, removing B replaces AB and BC by AC. The saving is

$$\Delta L = \|B - A\| + \|C - B\| - \|C - A\|.$$

For coordinates $A = (0, 0)$, $B = (1, 1)$ and $C = (2, 0)$ in metres, the old length is $2\sqrt{2}$ m and the shortcut is 2 m. The saving is about 0.828 m. A positive saving alone is insufficient: an obstacle may block the shortcut.

Separate example – three possible removals

For a different current path, suppose the checks give these options:

- Remove waypoint 1: saving 0.83 m, replacement segment allowed.
- Remove waypoint 2: saving 1.10 m, replacement segment blocked.
- Remove waypoint 3: saving 0.20 m, replacement segment allowed.

Choose waypoint 1, not the blocked option with the largest saving. After each deletion, scan the current path again and recalculate savings. Do not simply remove the first allowed point or sort the original savings once. Stop when no allowed deletion saves more than 10^{-12} m. A locally improved route is not guaranteed to be globally shortest.

9 Practise by changing one thing at a time

These short activities are optional. Choose the skill you want to practise using the supplied files. They are not a new graded assignment or a set of examination questions. Keep an unmodified copy so that you can compare results. No report, screenshot collection or new repository is required.

1. Trace a calculation and detect a wrong unit conversion

In `functions_review.py`, first add calls with `250.0` and `2.5`. The expected metre values are `0.25` and `0.0025`. Save and run the file. Then temporarily change the divisor from `1000.0` to `100.0` and run the selected tests. Identify which expected value is violated. Restore the correct conversion and rerun the tests.

2. Change an array operation and predict its shape

In `arrays_review.py`, change the displacement to `[2.0, 1.0]`. The first translated row should be `[4.0, 5.0]`. Explain why every row changes but the source array does not. Compare the printed reductions with `axis=0` and `axis=1`. Then change `view = shared_source[1:2]` to `view = shared_source[1:2].copy()`. After the edit through `view`, the original middle row should stay unchanged.

3. Change the search resolution, not the selection rule

In `models_review.py`, change each grid axis from three values to five values over the same interval. Check that there are `25` pairs and that the spacing becomes `0.5`. The first and last values on each axis should remain `-1.0` and `1.0`. Explain why generating more pairs does not by itself guarantee a feasible or optimal route.

Run supplied checks – Anaconda Prompt, week07_review

The supplied tests describe the original examples. Restore deliberate changes before using this full-suite result as a check of the supplied version.

```
python -m pytest -q
```

Output

```
.........................                                                [100%]
27 passed in 0.11s
```

Passing these checks confirms their stated cases, not every possible input or physical situation. When an edit fails, work back through the same chain: the actual input, the object or array holding it, the operation, the returned value and the assertion. That habit connects the six weeks of programming work more reliably than memorizing one output.

9.1 Write a class for calibrated measurements

These two optional exercises use unfinished files in `practice`. Write the class, connect its objects and check the results. They review Sections 2–4, with no new graded assignment or submission. The five demonstration files remain complete.

A sensor gives raw numerical readings. A supplied linear rule converts each reading into a length:

$$\text{corrected length (mm)} = \text{gain} \times \text{raw reading} + \text{offset (mm)}.$$

The gain is in millimetres per sensor unit. Its value and the offset are given, not estimated. Inputs are valid finite numbers and the calculations remain finite. No device connection or input-validation code is required.

Exercise 1 – calibration.py

Complete the two marked regions in `practice/calibration.py`.

1. `E1_CLASS`: complete `LinearCalibration`. Define the class attribute `output_unit = "mm"`. Write `__init__(self, gain, offset_mm)`. Store each object's coefficients as `self.gain` and `self.offset_mm`.
2. `E1_APPLY`: use the current instance attributes and `raw` to calculate `corrected_mm` with the rule above. Keep `return corrected_mm`.

Replace only the indicated regions. Keep the method names, parameters, surrounding code and supplied returns. Save the file before running it. The supplied demonstration calls `LinearCalibration(2.0, 1.0).apply(3.0)` and `LinearCalibration(0.5, -2.0).apply(8.0)`. These must give different results using each object's own coefficients.

Run – Anaconda Prompt, from week07_review

```
cd practice
python calibration.py 1
```

Output

```
unit: mm
corrected: 7.0
unit: mm
corrected: 2.0
```

Check – stay in practice

```
python -m pytest check_calibration.py -q -k calibration_case
```

After Exercise 1, expect `2 passed, 4 deselected`. Exercise 2 may still be unfinished. Before editing, these two checks fail with `NotImplementedError`, not an import or installation error.

9.2 Connect the objects and check their behaviour

Exercise 2 – calibration.py and check_my_measurements.py

A `MeasurementSet` holds raw readings and uses a `LinearCalibration` object to convert them. Complete Exercise 1 first. The readings are a nonempty list or tuple of valid finite numbers.

In `calibration.py`, complete the existing `MeasurementSet` class:

1. `E2_INIT`: store the *same* supplied calibration object as `self.calibration`. Store `tuple(readings)` as `self.raw`, so later edits to the caller's list do not change the stored values.
2. `E2_VALUES`: call `self.calibration.apply(raw)` for each value in `self.raw`, preserving order. Use a loop or comprehension to build a fresh list `values_mm` on every call. Keep `return values_mm`. Use the connected object's current coefficients. Do not repeat its formula, print results or change `self.raw`.

The two cases use `[1.0, 3.0]` with gain `2.0`, offset `1.0`, and `[4.0, 8.0]` with gain `0.5`, offset `-2.0`.

Run – Anaconda Prompt, practice folder

```
python calibration.py 2
```

Output

```
same calibration: True
stored raw: (1.0, 3.0)
corrected: [3.0, 7.0]
same calibration: True
stored raw: (4.0, 8.0)
corrected: [0.0, 2.0]
```

Write your assertions – check_my_measurements.py

The supplied `test_connection_and_ownership()` uses gain `2.0`, offset `1.0` and source `[1.0, 3.0]`. It creates `measurements`, changes `source[0]` to `99.0`, and assigns `result = measurements.corrected()`. Replace its `E2_ASSERT` region with these checks:

1. Assert that `measurements.calibration is calibration`.
2. Assert that `measurements.raw` is still `(1.0, 3.0)` after the source-list edit, and that `result` equals `[3.0, 7.0]`.
3. Set `result[0] = 99.0`. Assert that a new `measurements.corrected()` call still returns `[3.0, 7.0]`.

Exact equality is suitable for these exactly representable values. For general floating-point comparisons, use the explicit tolerance discussed in Section 2.3.

Check – Anaconda Prompt, practice folder

```
python -m pytest check_calibration.py check_my_measurements.py -q
```

Expected summary after both exercises: `7 passed`.
Before editing, these seven checks fail with `NotImplementedError`. They are separate from the 27 demonstration tests. Use `cd ..` to return to `week07_review`.